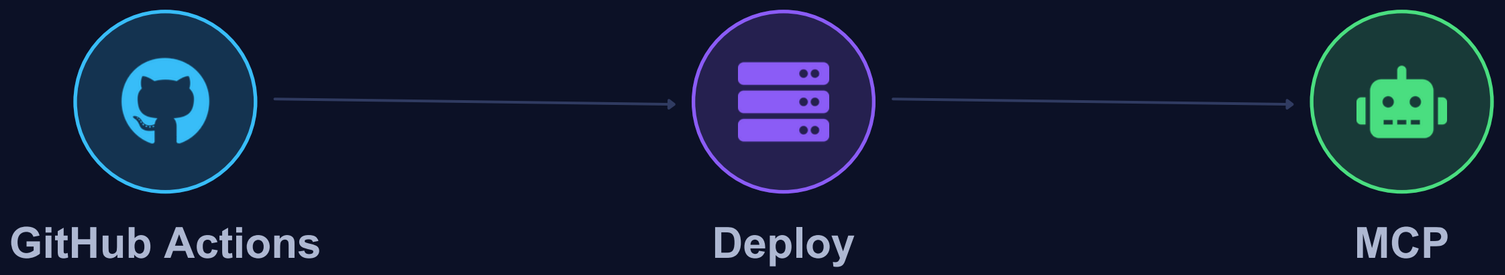
DIA 2 · MÃO NA MASSA

Do Código à Produção Pipeline, deploy e IA operando o serviço

FELIPE S.R. MACEDO



ROTEIRO DO DIA 2

Hoje o Suporte Bot vai para produção

01

Local e container

subir a API e empacotar com Docker

02

GitHub Actions

testes e build automáticos a cada push

03

Deploy na VM

staging e produção via SSH

04

Integração de IA

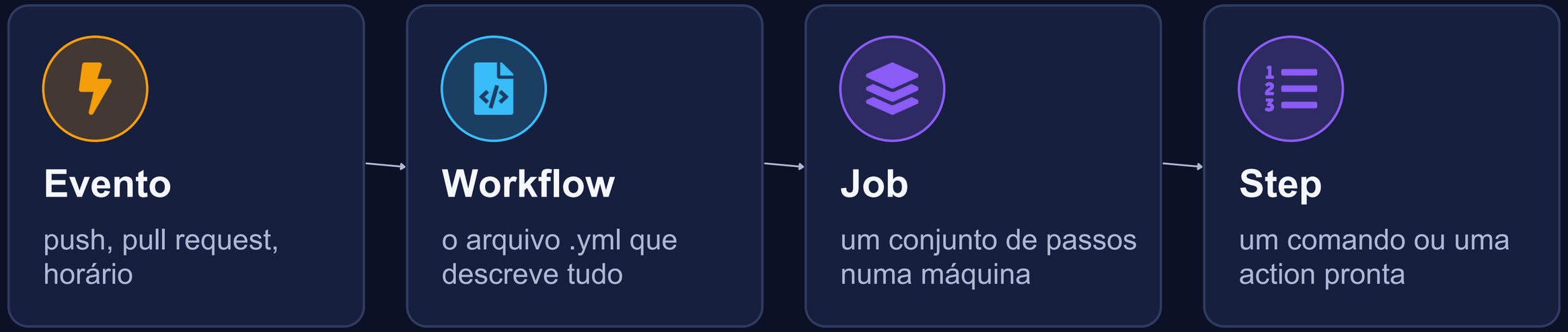
MCP ao vivo com o Claude



Travou? Cada bloco tem uma branch pronta. git switch etapa-N e siga com a turma.

Um robô que trabalha a cada push

O GitHub Actions executa automações descritas em arquivos YAML dentro do repositório, numa máquina emprestada pelo GitHub.



Runner: a máquina que executa o job. Usamos ubuntu-latest, criada do zero a cada execução e destruída no fim.

Anatomia do nosso workflow

.github/workflows/ci.yml

```yaml
name: CI
on: [push, pull_request]
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: ruff check .
      - run: pytest
      - run: docker build -t suporte-bot .
```

`on`	quando o workflow dispara
`runs-on`	em qual máquina ele roda
`uses`	reaproveita uma action pronta
`run`	executa um comando no terminal

O portão de qualidade



Tudo verde

A imagem vai para o registry e o PR pode entrar na main.

Um passo falhou

A pipeline para ali. Nada é publicado e o merge fica bloqueado até corrigir.

Segredos e a imagem no GHCR

publicar a imagem

```
permissions:
  packages: write
steps:
  - uses: docker/login-action@v3
    with:
      registry: ghcr.io
      username: ${{ github.actor }}
      password: ${{ secrets.GITHUB_TOKEN }}
  - run: docker push \
      ghcr.io/<usuario>/suporte-bot:${{ github.sha }}
```



Secrets
Settings → Secrets: o valor nunca aparece nos logs



GITHUB_TOKEN
criado sozinho a cada execução



Tag = commit
cada imagem leva o SHA do commit

CHECKPOINT badge de CI verde no README e a imagem aparecendo em Packages no GitHub.

Preparando a VM antes do primeiro deploy

Usuário deploy
sem root, com sudo

```
$ sudo adduser deploy
```

Login por chave
teste antes de tirar a senha

```
$ ssh-copy-id deploy@<ip>
```

Firewall
só SSH, 80 e 443

```
$ sudo ufw allow OpenSSH
```

Fail2Ban
IP da sala no ignoreip

```
$ sudo apt install fail2ban
```

Docker
o deploy usa compose

```
$ docker compose version
```

Pasta do app
compose.yaml + .env

```
$ mkdir ~/app
```

Como o deploy acontece



GitHub Actions
CI verde

SSH
chave nos Secrets

VM Linux
puxa a imagem nova

/health
respondeu? pronto

job de deploy

```
- name: deploy
  run: |
    echo "${{ secrets.SSH_KEY }}" > key && chmod 600 key
    ssh -i key deploy@${{ secrets.VM_HOST }} \
      "cd ~/app && docker compose pull && docker compose up -d"
```

Staging primeiro, produção depois

Staging
a cada pull request

- compose.staging.yaml
- porta 8001
- chave de API de teste

Produção
a cada merge na main

- compose.yaml
- porta 8000
- chave de API real

Mesma imagem nos dois. Só muda a configuração — o que você testou em staging é exatamente o que vai para o cliente.

Deu ruim? Rollback em um comando

Como cada imagem tem a tag do commit, voltar atrás é só apontar para a tag anterior e subir de novo.

a41f9c2

ontem

7be03d1

hoje 09:12

e19c8aa

hoje 10:40 · quebrou

volta para a versão boa

rollback

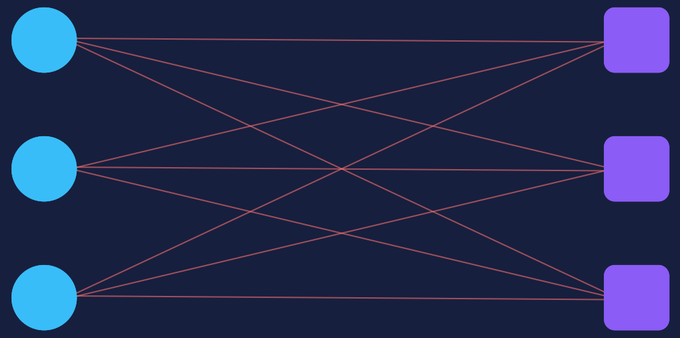
```
# o compose usa a imagem :${TAG}
$ cd ~/app
$ export TAG=7be03d1
$ docker compose up -d
$ curl localhost:8000/health
```

CHECKPOINT URLs de staging e produção respondendo /health.

Por que o MCP existe

ANTES

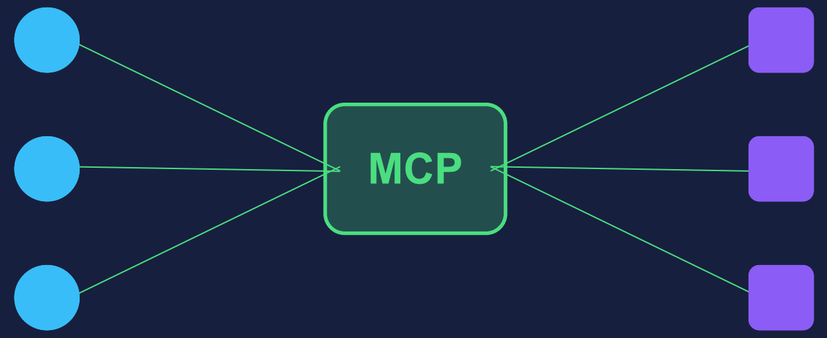
Cada IA precisava de uma integração feita sob medida para cada sistema.



3 IAs × 3 sistemas = 9 integrações

COM MCP

Cada sistema ganha um servidor MCP uma vez, e qualquer IA compatível usa.



um padrão, encaixa em todos

● IAs ■ sistemas

As peças do MCP



O QUE UM SERVIDOR MCP OFERECE

Tools

ações que a IA pode executar

`abrir_chamado`

Resources

dados que a IA pode ler

`faq://planos`

Prompts

instruções prontas para reusar

`triagem_suporte`

Transporte: stdio (servidor local, no mesmo computador) ou Streamable HTTP (servidor remoto).

O servidor MCP do Suporte Bot

mcp_server.py

```python
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("suporte-bot")

@mcp.tool()
def consultar_status_conexao(cliente: str) -> str:
    """Diz se a conexão do cliente está ativa."""
    return api.status(cliente)
@mcp.tool()
def abrir_chamado(cliente: str, motivo: str) -> str:
    """Abre um chamado técnico e devolve o número."""
    return api.novo_chamado(cliente, motivo)

mcp.run()
```



@mcp.tool()
vira uma ferramenta para a IA



A docstring
é como a IA sabe quando usar



Os tipos
viram o formato dos argumentos

Ao vivo: o Claude operando o serviço

01 `tools/list`
O Claude pergunta quais ferramentas existem

02 `decide`
Lê o pedido e escolhe consultar_status_conexao

03 `tools/call`
Chama a ferramenta com o número do cliente

04 `tools/call`
Conexão caída: abre o chamado em produção

O PEDIDO NA TELA

“O cliente 4021 diz que está sem internet desde cedo. Verifica e, se precisar, abre um chamado.”

Enquanto o Claude age, o terminal mostra cada mensagem JSON-RPC indo e voltando.

IA em produção exige cuidado



Custo

cada chamada ao LLM é cobrada

Como evitar: limite de gasto e cache



Vazamento de dados

o que vai no prompt sai da sua rede

Como evitar: envie só o necessário



Prompt injection

texto malicioso dá ordens à IA

Como evitar: ferramentas com permissão mínima



Alucinação

resposta convincente e errada

Como evitar: evals e confirmação humana

ENCERRAMENTO

O que você levou para casa



PARA CONTINUAR ESTUDANDO

Terraform
infraestrutura como código

Kubernetes
orquestrar vários serviços

LLMOps
versionar prompts e evals

Segurança
scan de imagens e SBOM

FIM DA OFICINA

Obrigado!

Perguntas, feedback e o formulário de avaliação.

FELIPE S.R. MACEDO